



Radiografía de tus Aplicaciones: Observabilidad con **Micrometer**

Llevando métricas de **Spring Boot** y **Quarkus** hasta **Grafana**

El Problema de la Caja Negra en Producción

¿Cuántas transacciones por segundo estamos procesando?

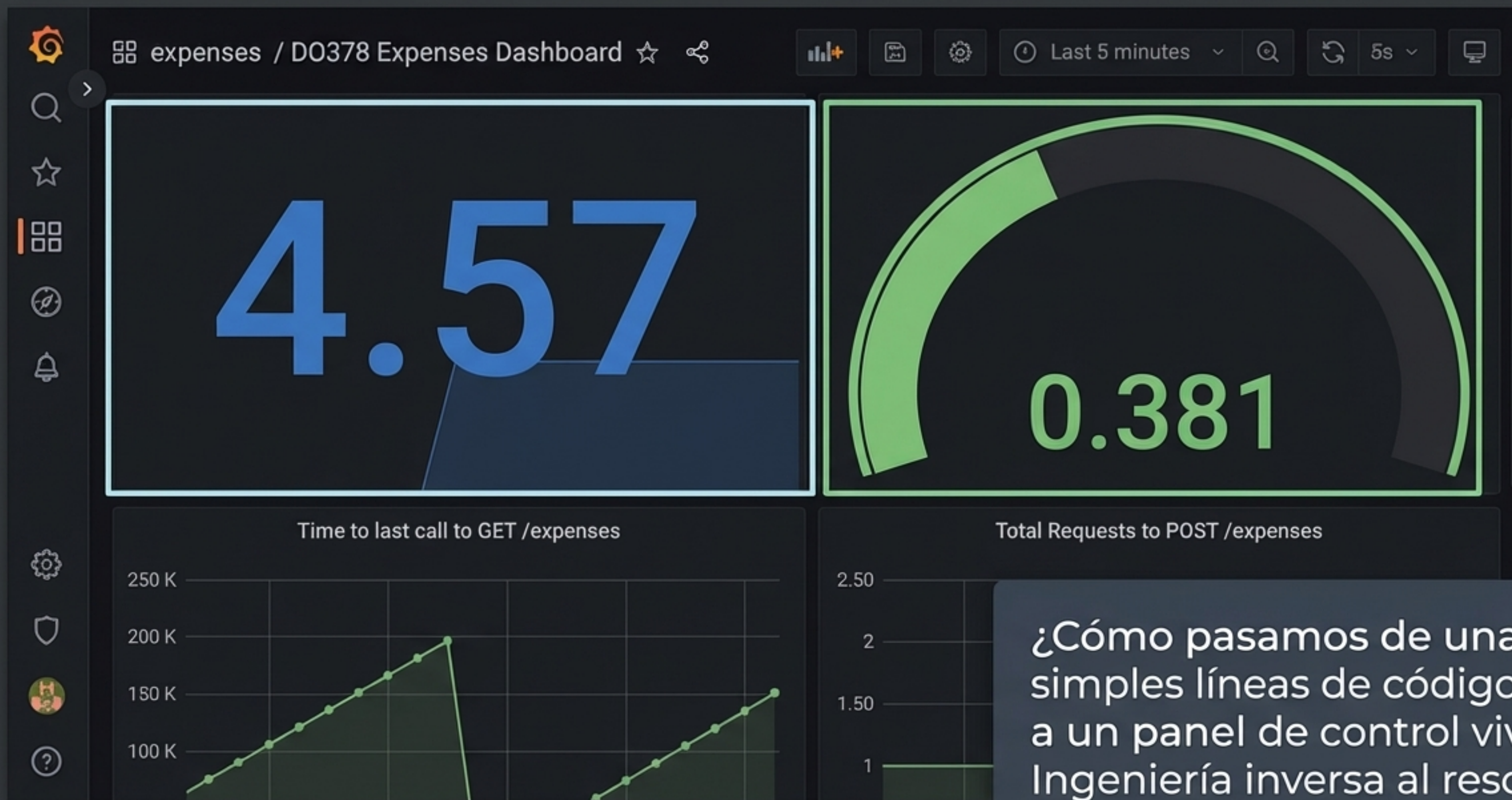


¿Cuánto tiempo tardan realmente las consultas a la base de datos?

¿Nos estamos quedando sin memoria bajo carga?

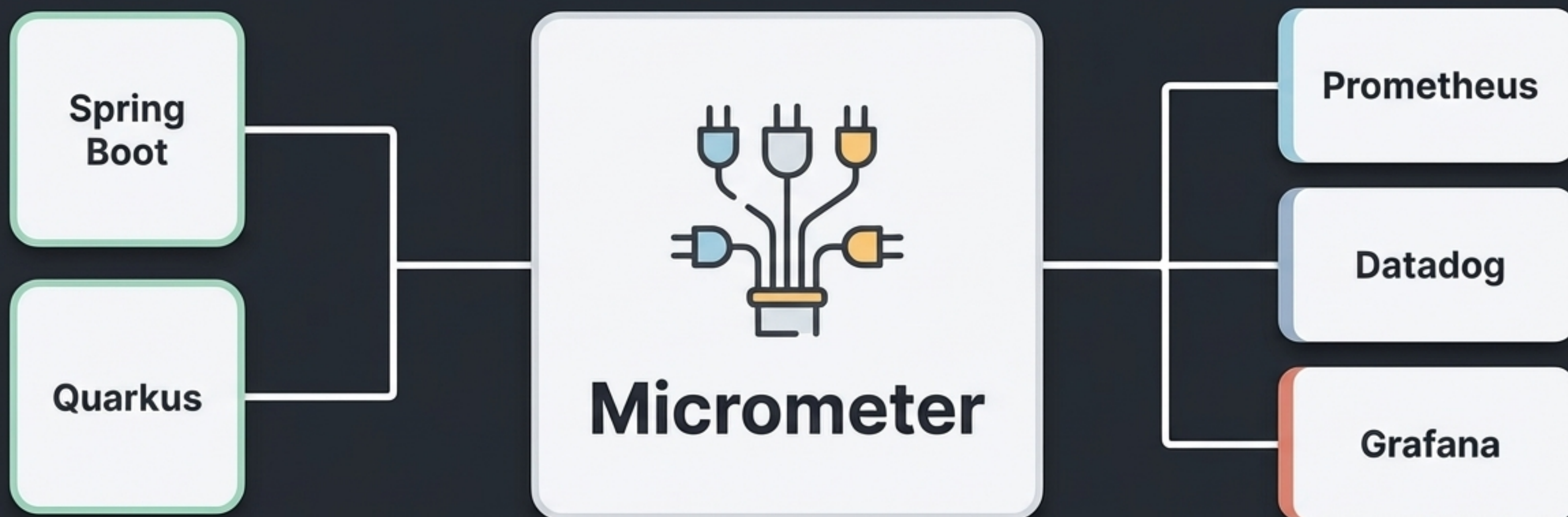
Tu código compila y se ejecuta. Pero sin observabilidad, estás operando a ciegas.

El Destino: Observabilidad Total en Tiempo Real



¿Cómo pasamos de unas simples líneas de código Java a un panel de control vivo? Ingeniería inversa al rescate.

Introduciendo Micrometer: El Adaptador Universal



Micrometer es el SLF4J de las métricas. Escribes la instrumentación una sola vez en tu código Java, y Micrometer se encarga de traducirla al formato de cualquier sistema de monitoreo moderno.

El Centro de Mando: MeterRegistry



```
package com.redhat.training;

import io.micrometer.core.instrument.MeterRegistry;

public class MyResource {
    @Inject
    public MeterRegistry registry;
    .code omitted...
```

En Quarkus usamos **@Inject**. Si estás en Spring Boot, usarías **@Autowired**.

El componente central. Es el registro en memoria donde vivirán todos los contadores y cronómetros de tu aplicación.

Los 3 Pilares de la Medición (Matriz Diagnóstica)



Counters (Contadores)

- **Definición:** Eventos discretos que solo pueden incrementar.
- **Caso de uso:** Total de compras realizadas, Errores 500 generados.



Gauges (Calibradores)

- **Definición:** Un valor actual que puede subir o bajar en cualquier momento.
- **Caso de uso:** Usuarios activos en este momento, Memoria RAM en uso.



Timers (Temporizadores)

- **Definición:** Mide la duración y la frecuencia de eventos cortos.
- **Caso de uso:** Latencia de una petición HTTP, Tiempo de ejecución de una query.

Counters: Registro e Incremento Manual

```
package com.redhat.training;
import io.micrometer.core.instrument.MeterRegistry;
import io.micrometer.core.instrument.Tags;
import javax.annotation.PostConstruct;
import javax.enterprise.context.ApplicationScoped;
import javax.inject.Inject;
import javax.ws.rs.GET;
import javax.ws.rs.Produces;
import javax.ws.rs.core.MediaType;
import java.util.Set;
@Path("/card-transactions")
@ApplicationScoped
public class CardTransactionsResource {
    @Inject
    MeterRegistry registry;

    @PostConstruct
    public void initMeters() {
        registry.counter(
            "card_transaction",
            Tags.of("description", "Count of card transactions")
        );
    }

    @GET
    @Produces(MediaType.APPLICATION_JSON)
    public Set<CardTransaction> list() {
        registry.counter("card_transaction").increment();
        // ...code omitted...
        return transactionsService.list();
    }

    // ...code omitted...
}
```

1

Fase 1: Configuración.

Definir y nombrar el contador en el registro durante la inicialización (@PostConstruct).

2

Fase 2: Acción.

Llamar a .increment() cada vez que el evento ocurre (ej. cuando se lista una transacción).

Counters: La Elegancia de las Anotaciones

```
    @Counted(  
        value = "card_transaction",  
        description = "Count of card transactions"  
    )  
    public Set<CardTransaction> list() {  
        return transactionsService.list();  
    }
```

Insight Box: Menos código repetitivo, mismo resultado. Al decorar el método con `@Counted`, el framework intercepta la llamada y cuenta automáticamente cada ejecución exitosa, manteniendo tu lógica de negocio completamente limpia.

Gauges: Observando el Estado Actual

```
package com.redhat.training;

import io.micrometer.core.instrument.MeterRegistry;

...code omitted...
public class CardTransactionsResource {
    private final Stopwatch stopWatch = Stopwatch.createStarted(); ❶

    @Inject
    public MeterRegistry registry;

    @PostConstruct
    public void initMeters() {
        registry.gauge(❷
            "time_since_last_exec", ❸
            Tags.of("description", "Time since the last execution"), ❹
            stopWatch, ❺
            Stopwatch::getTime ❻
        );
    }

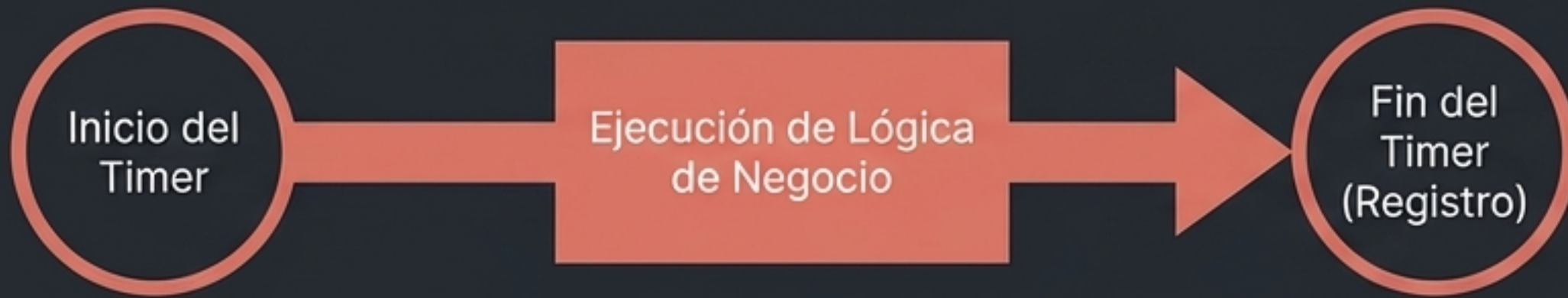
    @GET
    public Set<CardTransaction> list() {
        stopWatch.reset(); ❼
        stopWatch.start(); ❽
    }
}
```



A diferencia del Contador que nosotros incrementamos manualmente, a un Gauge le entregamos una función (ej. `Stopwatch::getTime`). Micrometer 'observará' esta función periódicamente para leer el valor actual.

Timers: Midiendo el Rendimiento

```
● ● ●  
  
@POST  
public Transaction create(CardTransaction trx) {  
    return registry.timer("transaction_creation_time")  
        .wrap(  
            (Supplier<Transaction>) () -> transactionsService.create(trx)  
        ).get();  
}
```



La función `.wrap()` es un envoltorio mágico. Ejecuta tu bloque de código y, simultáneamente, cronometra cuánto tarda de principio a fin, registrando el tiempo total, el promedio y los picos máximos.

La Exposición de Datos: Spring Boot vs. Quarkus

Spring Boot

Dependencia

```
spring-boot-starter-actuator
```

Endpoints

```
/actuator/metrics  
/actuator/prometheus
```

Quarkus

Extensión

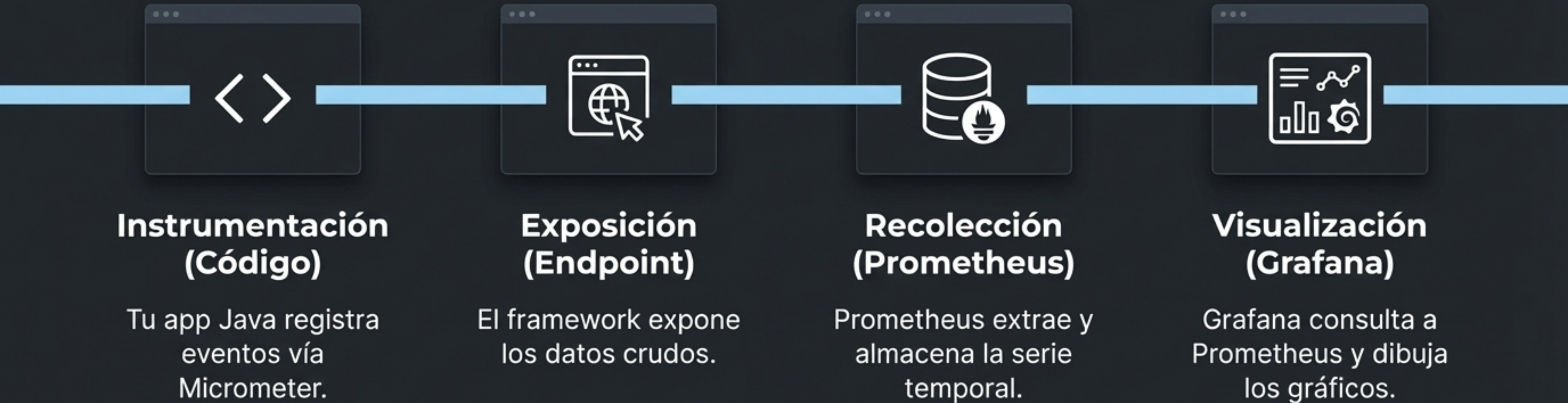
```
quarkus-micrometer-registry-prometheus
```

Endpoint

```
/q/metrics
```

Ambos frameworks exponen los datos en el formato estándar de texto plano que Prometheus utiliza para hacer el 'scraping' (recolección).

Síntesis: El Pipeline de la Observabilidad



De la Teoría a la Producción

El Estándar

Usa Micrometer como tu API definitiva. Desacopla tu código del sistema de monitoreo.

La Herramienta Correcta

Selecciona con precisión: Contadores para sumas, Gauges para estados actuales, y Timers para rendimiento.

Código Limpio

Prioriza las anotaciones (@Counted, @Timed) sobre el registro manual para mantener la arquitectura limpia.

Siguiente paso: Añade la extensión micrometer a tu próximo proyecto universitario y observa tu código cobrar vida.